

ISCTE – Instituto Universitário de Lisboa
Escola de Tecnologias Digitais Aplicadas

Atelier WEC:

Loja de roupa de luxo

Projeto da unidade curricular:
Projetos em Ambiente Web e Cloud

Docente: João Miguel Matos

Realizado por:
Rafaela Ferreira Nº 110826 – TDIA
Maria Costa Nº 130760 - TDIA

Ano letivo 2025/2026
Sintra, 2026

Índice

1. Introdução/ Objetivos.....	1
2. Análise e Planeamento	1
2.1 Enunciado e requisitos	1
2.2 Tecnologias utilizadas	1
Linguagens	1
Frameworks e bibliotecas	1
Serviços Cloud.....	1
3. Design da Solução	1
4. Implementação	1
Organização de pastas e ficheiros	1
Ficheiros de bootstrapping.....	1
Camada de estado global: StoreContext.jsx	1
Ficheiro de dados: storeData.js	1
Componentes de layout: Header e Footer	1
Header	1
Footer.....	1
Estilos globais e design system.....	1
Páginas principais (Pages).....	1
HomePage.jsx.....	1
CategoryPage.jsx.....	1
ProductPage.jsx	1
CartPage.jsx.....	1
FavoritesPage.jsx.....	1
CheckoutPage.jsx.....	1
AccountPage.jsx.....	1
SearchPage.jsx.....	1
InfoPage.jsx	1
Componentes reutilizáveis.....	1

ProductCard.jsx	1
ProductGrid.jsx	1
Pagination.jsx	1
CategoryToolbar.jsx	1
categoryFilters.jsx	1
FilterSidebar.jsx	1
Newsletter.jsx, FooterColumns.jsx, FooterBottom.jsx	1
Funcionamento global e fluxo de dados	1
5. Resultados e Discussão	1
Avaliação crítica	1
Limitações	1
Comparação com objetivos	1
5. Conclusões e Trabalho Futuro.....	1
Conclusões.....	1
Conclusões gerais	1
Melhorias possíveis	1
Referências.....	1

1. Introdução/ Objetivos

O presente relatório descreve o planeamento e desenvolvimento de uma Single Page Application (SPA) de comércio eletrónico de moda de luxo, concebida no âmbito da unidade curricular de Projetos em Ambientes Web e Cloud. O objetivo central consiste em aplicar metodologias modernas de arquitetura de front-end, procurando equilibrar desempenho técnico, organização modular do código e uma experiência de utilizador (UX) sofisticada e intuitiva.

A aplicação, designada Atelier WEC, foi implementada utilizando a biblioteca React e adota uma arquitetura baseada em rendering e navegação no lado do cliente. Em vez de recorrer a múltiplos documentos HTML independentes, a solução utiliza uma abordagem de SPA assente em navegação no browser, permitindo que a transição entre secções, a atualização do carrinho de compras, os sistemas de filtragem e outras interações ocorram de forma imediata, sem recarregar a página, o que proporciona uma experiência mais fluida e responsiva.

Ao longo do projeto foram implementadas funcionalidades essenciais de uma plataforma de comércio eletrónico moderna, incluindo catálogo dinâmico, categorização de produtos, pesquisa, filtros, carrinho, favoritos, checkout e área de conta, sempre com base em componentes reutilizáveis e independentes. Paralelamente, recorreu-se à gestão de estado global e à persistência de dados no navegador para simular autenticação de utilizadores e armazenamento de preferências, em linha com os objetivos pedagógicos da unidade curricular.

Em termos conceptuais, a aplicação pretende reproduzir uma montra digital de moda e lifestyle com identidade visual própria, incluindo secções para mulher, homem, criança, casa e páginas institucionais de apoio. Esta primeira entrega corresponde à parte inicial da solução global e tem como foco a consolidação da base front-end, da navegação e das principais interações com o utilizador, preparando o projeto para evoluções futuras ao nível de backend e serviços cloud.

2. Análise e Planeamento

A fase de análise e planeamento foi essencial para transformar os requisitos do enunciado numa solução técnica coerente, escalável e compatível com uma futura expansão da aplicação. Desde o início, optou-se por organizar o projeto em torno de componentes reutilizáveis, páginas de alto nível e uma camada central de estado global, de forma a evitar duplicação de lógica e facilitar a manutenção do código.

A estrutura definida permite separar claramente o que pertence à apresentação, à navegação, aos dados e à gestão de interações do utilizador. Esta decisão foi importante porque a aplicação inclui várias áreas com comportamento relacionado, por exemplo, o carrinho, os favoritos e a conta do utilizador, que precisam de partilhar estado sem depender de cadeias extensas de props.

Do ponto de vista de planeamento funcional, a aplicação foi pensada como um conjunto de fluxos principais: exploração do catálogo, consulta do detalhe de produto, adição ao carrinho, marcação de favoritos, autenticação simulada, pesquisa global e finalização de compra. Estes fluxos influenciaram diretamente a definição das rotas, das páginas e dos componentes auxiliares usados ao longo do projeto.

2.1 Enunciado e requisitos

A análise do enunciado levou à identificação de um conjunto de requisitos funcionais centrais para a primeira fase do projeto. Entre esses requisitos encontra-se a necessidade de implementar uma loja online responsiva com catálogo organizado por categorias, navegação entre páginas, filtragem e pesquisa de produtos, páginas individuais de detalhe e mecanismos de interação típicos de e-commerce.

Com base nos ficheiros entregues, é possível verificar que os principais requisitos funcionais já contemplados são os seguintes:

- Apresentação de uma homepage com secções de destaque, coleções e acesso às principais áreas do catálogo.
- Navegação por categorias e subcategorias, com rotas dinâmicas definidas em `App.jsx`.
- Listagem de produtos em grelha, através de componentes reutilizáveis como `ProductGrid` e `ProductCard`.

- Página de detalhe de produto com seleção de tamanho, adição ao carrinho e gestão de favoritos.
- Carrinho de compras com atualização de quantidades, remoção de itens e cálculo de total.
- Página de favoritos associada ao estado do utilizador autenticado.
- Área de conta com registo, login, logout e persistência local de dados do utilizador.
- Página de checkout com formulário e resumo da encomenda.
- Página de pesquisa para localizar produtos por texto.
- Páginas informativas e institucionais ligadas ao footer.

A nível não funcional, o projeto também evidencia preocupação com usabilidade, consistência visual, organização modular do código e adaptação a diferentes tamanhos de ecrã. A utilização de media queries, `clamp()` e grelhas responsivas mostra que a responsividade não foi tratada como detalhe posterior, mas integrada diretamente no desenho da interface.

2.2 Tecnologias utilizadas

A solução foi desenvolvida recorrendo a um conjunto de tecnologias adequadas ao desenvolvimento de interfaces modernas baseadas em componentes.
`main.jsx+2`

Linguagens

As linguagens principais utilizadas foram:

- **JavaScript**, como linguagem base de desenvolvimento da lógica da aplicação e dos componentes React.App.
- **JSX**, como extensão sintática usada para descrever a estrutura declarativa da interface dentro dos componentes.
- **CSS**, utilizado para toda a componente visual da aplicação, incluindo layout, tipografia, cores, responsividade e efeitos de interação.

Frameworks e bibliotecas

As principais bibliotecas/frameworks observáveis no projeto são:

- **React**, utilizado para a construção da interface em componentes, organização da aplicação por páginas e atualização reativa da UI.
`main.jsx+2`

- **React Router DOM**, responsável pela navegação client-side e pela definição de rotas como home, produto, carrinho, conta, favoritos, pesquisa e checkout.
- **Context API**, implementada em `StoreContext.jsx`, para centralizar o estado global da loja e evitar prop drilling.
- **Web Storage API** (`localStorage` e `sessionStorage`), utilizada para persistir sessões, utilizadores, carrinho de utilizador convidado e informação auxiliar de navegação.

Serviços Cloud

Nesta primeira fase, não se observa integração com serviços cloud externos para base de dados, autenticação ou pagamentos reais. Em vez disso, o projeto utiliza armazenamento local no navegador e uma fonte de dados estática (`storeData.js`), o que é adequado para um protótipo funcional de front-end e para uma primeira entrega focada em arquitetura, interface e comportamento no cliente.

Ainda assim, a estrutura atual permite uma futura integração com serviços cloud como Firebase, Supabase, Auth0 ou APIs REST próprias, uma vez que a lógica da interface já está separada da camada de dados.

3. Design da Solução

O design da solução foi orientado por três princípios fundamentais: modularidade, reutilização e separação de responsabilidades. Em vez de concentrar toda a lógica numa única página ou componente, a aplicação foi dividida em ficheiros especializados, cada um com uma função clara dentro da arquitetura global.

A navegação principal é orquestrada por `App.jsx`, que funciona como ponto central de roteamento e composição da interface. Este ficheiro associa cada URL à respetiva página e garante a presença contínua dos elementos estruturais transversais, nomeadamente o cabeçalho e o rodapé.

Ao nível dos dados, a solução assenta em duas peças fundamentais: `storeData.js`, que representa o catálogo estático de produtos e páginas temáticas, e `StoreContext.jsx`, que gere o estado dinâmico da aplicação. Esta divisão é particularmente importante porque evita misturar dados permanentes com estado mutável do utilizador, como carrinho, sessão autenticada e favoritos.

As páginas de alto nível (HomePage, CategoryPage, ProductPage, CartPage, FavoritesPage, CheckoutPage, AccountPage, SearchPage, InfoPage) foram concebidas como contentores funcionais que combinam vários componentes menores. Por sua vez, componentes como ProductCard, ProductGrid, CategoryToolbar, FilterSidebar, Pagination, Newsletter e FooterColumns asseguram reutilização, uniformidade visual e encapsulamento de responsabilidades específicas.

Em termos visuais, a solução segue uma identidade inspirada em marcas de luxo, recorrendo a fundos escuros, alguns detalhes dourados, tipografia serifada, organização de forma espaçada... Esta decisão de design contribui para diferenciar o projeto de uma loja genérica e reforça o conceito que procuramos com a marca Atelier WEC.

A responsividade também faz parte do design da solução e não apenas da implementação técnica. A interface utiliza CSS Grid, Flexbox, media queries e unidades fluidas para adaptar a disposição dos elementos em desktop, tablet e mobile, garantindo acessibilidade e legibilidade em diferentes contextos de utilização.

4. Implementação

A aplicação “Atelier WEC” foi desenvolvida como uma Single Page Application (SPA) em React, seguindo uma arquitetura baseada em componentes e em gestão de estado global através da Context API. Em vez de existir um conjunto de múltiplas páginas HTML independentes, a navegação é feita inteiramente no lado do cliente, com o React Router DOM a intercetar as alterações de URL e a renderizar dinamicamente o componente de página correto sem recarregar o documento.

A estrutura lógica pode ser pensada em três grandes camadas:

- Camada de apresentação (UI): componentes de interface reutilizáveis (cartões de produto, grelhas, filtros, formulários) e componentes de página que funcionam como “contentores” dessas peças.
- Camada de roteamento: centralizada em `App.jsx`, onde se definem as rotas principais (home, categorias, produto, carrinho, favoritos, checkout,

conta, pesquisa, informações) e se decide que página é iniciada para cada URL.

- Camada de gestão de estado: abstrata para o ficheiro `StoreContext.jsx`, que implementa um contexto global com o estado do carrinho, dos favoritos, dos filtros e outros dados transversais à aplicação, evitando “prop drilling” entre componentes intermédios.

O ficheiro `storeData.js` funciona como fonte de dados estática da loja, contendo o catálogo de produtos e as respetivas categorias/subcategorias, preços, cores e restantes propriedades, sobre o qual todas as outras funcionalidades operam (listagem, filtros, pesquisa, favoritos e carrinho).

Organização de pastas e ficheiros

A estrutura do projeto segue a organização típica de um front-end moderno em React com Vite, separando claramente:

- Ficheiros de arranque e configuração (`main.jsx`, `App.jsx`).
- Contexto e lógica de negócio (`StoreContext.jsx`).
- Páginas de alto nível (ficheiros `*Page.jsx`).
- Componentes reutilizáveis (cartões de produto, grelhas, paginação, filtros, header, footer).
- Estilos globais e específicos (`index.css`, `App.css`, `Header.css`, `Footer.css`).

Ficheiros de bootstrapping

- `main.jsx`: é o ponto de entrada da aplicação. Importa o React, o `BrowserRouter` do `react-router-dom`, o `StoreProvider` e o componente `App`. Envolve o `<App />` com o `<StoreProvider>` e com o `<BrowserRouter>`, garantindo, por um lado, a disponibilidade do contexto global em toda a árvore de componentes e, por outro, a existência de routing client-side.
- `App.jsx`: define o layout principal da SPA e a tabela de rotas da aplicação. Renderiza o `<Header />` no topo, o `<Footer />` no final da página e, entre ambos, um conjunto de `<Route>` que mapeiam cada caminho para o respetivo componente de página (por exemplo, `/ → HomePage`, `/category/:categorySlug → CategoryPage`, `/product/:id → ProductPage`, `/cart → CartPage`, `/favorites → FavoritesPage`,

/checkout → CheckoutPage, /account → AccountPage, /search → SearchPage, /info → InfoPage).

Desta forma, o App funciona como “shell” da interface, garantindo que o cabeçalho e o rodapé se mantêm constantes enquanto o conteúdo central é substituído conforme a rota ativa.

Camada de estado global: StoreContext.jsx

- `StoreContext.jsx`: implementa a Context API que centraliza o estado transacional do utilizador e toda a lógica de negócio associada ao catálogo. Aqui são mantidos, entre outros:
 - Lista completa de produtos, carregada a partir de `storeData.js`.
 - Conteúdo do carrinho de compras (itens, quantidades, tamanhos selecionados).
 - Lista de produtos favoritos (wishlist).
 - Estado de filtros ativos por categoria (ex.: tamanhos, cores, faixas de preço).
 - Informação de pesquisa e possivelmente dados de conta/checkout simples.

Além do estado, este contexto expõe funções como `addToCart`, `removeFromCart`, `updateQuantity`, `toggleFavorite`, `applyFilters`, `clearFilters` e outras operações de alto nível, que podem ser invocadas pelas páginas e componentes sem necessidade de encadear props. Esta abordagem concretiza o padrão Provider/Consumer, em que o `StoreProvider` envolve o App e todos os componentes filhos consomem o contexto através de hooks como `useContext(StoreContext)`.

Ficheiro de dados: storeData.js

- `storeData.js`: contém a estrutura de dados local que representa o inventário da loja. Define, por exemplo, arrays de produtos com campos como `id`, `name`, `category`, `subcategory`, `description`, `price`, `color`, `sizes`, `imageUrl` e outros dados utilizados tanto para a criação dos cartões como para a lógica de pesquisa e filtragem.

As páginas e componentes não vão diretamente à rede buscar dados; em vez disso, leem e filtram estes dados locais via contexto, o que é coerente com o objetivo de um projeto académico com foco em SPA e interface de luxo.

Componentes de layout: Header e Footer

Header

- `Header.jsx`: componente responsável pela barra de navegação principal da aplicação.
 - Logótipo ou nome da marca (Atelier WEC).
 - Ligações de navegação para as secções principais (Home, categorias Homem/Mulher/Criança, Pesquisa, Conta, etc.).
 - Acesso rápido ao carrinho de compras e à lista de favoritos, normalmente com ícones que mostram contadores dinâmicos de itens, consumindo o estado global do contexto.
- `Header.css`: ficheiro dedicado aos estilos do cabeçalho, garantindo um design consistente com o restante sistema (fundo escuro, tipografia elegante, linhas finas, ícones alinhados com o tema de luxo).

Footer

- `Footer.jsx`: componente que agrupa o rodapé da aplicação. Este componente funciona como contentor de peças mais pequenas:
 - `FooterColumns.jsx`: define as várias colunas do rodapé (por exemplo, “Ajuda”, “Conta”, “Informação Legal”), com listas de links e texto institucional.
 - `Newsletter.jsx`: secção para subscrição de newsletter, com um pequeno formulário (campo de email e botão) estilizado em coerência com o tema de luxo da interface.
 - `FooterBottom.jsx`: parte inferior do rodapé, usada para mostrar direitos de autor, informação de métodos de pagamento e outros detalhes de copyright.
- `Footer.css`: ficheiro que recolhe a definição visual do rodapé, incluindo cores, espaçamentos, alinhamentos das colunas e comportamento responsivo em ecrãs mais pequenos.

Deste modo, o Footer é também um exemplo claro do uso de micro-componentes reutilizáveis para construir uma secção mais complexa.

Estilos globais e design system

- `index.css`: contém resets globais e definição de tipografia base, cores de fundo e regras gerais para o `body`, links e elementos HTML principais.
- `App.css`: implementa o design system da interface, definindo variáveis CSS (custom properties) para cores, espaçamentos e larguras máximas, bem como estilos das principais secções da página (hero, grelhas de produtos, páginas de conta, carrinho, checkout, filtros, etc.).

Neste ficheiro estão concentradas as classes responsáveis por:

- Hero da Home e das categorias (`.home-hero`, `.category-hero`).
- Grelhas de produtos (`.product-grid`, `.summer-grid`).
- Cartões de produto e coleções (`.product-card`, `.collection-card`).
- Layouts de páginas (`.cart-page`, `.checkout-page`, `.account-page`, `.search-page`, `.info-page`).
- Componentes de interação: botões, controlo de quantidade, filtros, sidebar de filtros, formulário de checkout, etc.

O design utiliza um esquema de cor escura com contrastes em dourado (por exemplo, fundo aproximado a `#101010` e acentos em `#d4af37`), tipografia serifada para títulos e sans-serif para texto corrido, além de transições suaves para hovers e micro-interações. A responsividade é assegurada por media queries e por funções como `clamp()` em tamanhos de fonte e espaçamentos, tal como é pedido no enunciado (mobile-first e adaptação fluida a diferentes resoluções).

Páginas principais (Pages)

As páginas são componentes de alto nível que representam ecrãs completos e são mapeadas diretamente para rotas específicas.

HomePage.jsx

- `HomePage.jsx`: página inicial da aplicação.
 - Secção hero com headline e botões de ação para explorar o catálogo.
 - Secções de coleções de destaque ou “novidades”, construídas com componentes como `ProductGrid` ou grelhas específicas.

- Esta página consome dados de `storeData.js` através do contexto, podendo mostrar subconjuntos específicos do catálogo (por exemplo, “Summer Collections”, “Best Sellers”).

CategoryPage.jsx

- `CategoryPage.jsx`: responsável pela listagem de produtos de uma dada categoria principal (Homem, Mulher, Criança) ou subcategoria (Casacos, Calçado, Acessórios). É aqui que se combinam:
 - `CategoryToolbar.jsx`: barra de ferramentas no topo, onde o utilizador pode selecionar subcategorias, ordenar produtos ou abrir o painel de filtros.
 - `FilterSidebar.jsx`: componente lateral que apresenta filtros detalhados (tamanho, cor, preço, materiais, etc.) com a possibilidade de expandir/contrair grupos de filtros e de limpar todos os critérios.
 - `ProductGrid.jsx`: grelha reutilizável que recebe uma lista de produtos filtrados e a apresenta em formato de cartões.
 - `Pagination.jsx`: componente que permite paginar resultados quando existem muitos produtos, controlando página atual e número de itens por página.

Esta página é central para o requisito de “Catálogo Dinâmico e Categorização”, pois lê dinamicamente o catálogo a partir de `storeData.js` e adapta-se ao `categorySlug` presente no URL para determinar que subset de produtos deve ser mostrado.

ProductPage.jsx

- `ProductPage.jsx`: página de detalhe de produto, acessível através de uma rota com parâmetro (`/product/:id`). As principais responsabilidades incluem:
 - Obter o produto correto a partir do ID, usando os dados do contexto (`storeData.js`).
 - Apresentar imagens, nome, descrição, categoria, cor e preço, de forma destacada e coerente com o tema visual.
 - Permitir escolher tamanho (através de um “size picker”) e adicionar o produto ao carrinho (`addToCart`), bem como marcá-lo como favorito (`toggleFavorite`).

Esta página concretiza a ligação entre catálogo e carrinho, funcionando como uma ponte entre exploração e transação.

CartPage.jsx

- `CartPage.jsx`: representa o carrinho de compras. A página lista os produtos atualmente no carrinho, mostrando:
 - Imagem, nome, tamanho selecionado e preço unitário de cada item.
 - Controlo de quantidade com botões de increment/decrement e atualização do total por linha.
 - Botão para remover itens individuais ou esvaziar o carrinho.
 - Resumo do pedido com cálculo do subtotal e das taxas de envio, incluindo a aplicação da regra de isenção de portes quando o total atinge um determinado limiar (por exemplo, 150€).

Ao alterar quantidades ou remover produtos, o estado global é imediatamente atualizado e a interface reflete essas alterações em tempo real.

FavoritesPage.jsx

- `FavoritesPage.jsx`: página dedicada à wishlist do utilizador. Mostra:
 - Lista de produtos que foram marcados como favoritos, permitindo navegar para o detalhe ou, em alguns casos, adicionar diretamente ao carrinho.
 - Mensagens adequadas quando ainda não existem favoritos, incentivando a explorar o catálogo.

Esta página utiliza o estado de favoritos mantido no `StoreContext`, concretizando o requisito de “Gestão de Favoritos (Wishlist)”.

CheckoutPage.jsx

- `CheckoutPage.jsx`: implementa o fluxo de finalização de compra, em duas áreas principais: formulário de dados de faturação/entrega e sumário do pedido.
 - No formulário, o utilizador preenche campos como nome, email, morada, país, NIF, método de pagamento, entre outros. Todos estes campos são estilizados com as classes definidas em `App.css` para manter o aspeto premium.
 - No painel de resumo, a página volta a apresentar a lista de produtos no carrinho com subtotais, portes e valor total, permitindo confirmar visualmente a encomenda antes de submeter.

Embora o projeto atualmente não integre um gateway de pagamentos real, a `CheckoutPage` simula o comportamento de um checkout típico, alinhado com o objetivo pedagógico do enunciado.

AccountPage.jsx

- `AccountPage.jsx`: página de área de cliente, que simula um sistema de autenticação e gestão de perfil. As suas principais funcionalidades incluem:
 - Tabs de login e registo, com formulários distintos para cada operação.
 - Validação simples dos campos, feedback de erros e mensagens informativas.
 - Persistência de dados de utilizador no `localStorage` ou `sessionStorage`, de forma a simular uma “base de dados” local de contas, conforme sugerido pelo enunciado.

Depois de autenticado, o utilizador pode visualizar e editar dados de conta básicos e, eventualmente, encerrar sessão (logout), o que limpa o estado correspondente.

SearchPage.jsx

- `SearchPage.jsx`: componente que implementa o motor de pesquisa global da loja. Através de um campo de entrada (search bar), recebe uma query do utilizador e filtra o catálogo com base em vários campos:
 - Nome do produto.
 - Categoria e subcategoria.
 - Descrição e outros metadados relevantes.

Os resultados da pesquisa são mostrados numa grelha de produtos, reutilizando `ProductGrid` e `ProductCard`, permitindo manter o mesmo padrão visual em toda a aplicação.

InfoPage.jsx

- `InfoPage.jsx`: página institucional, dedicada a apresentar informação de contexto sobre a marca, o conceito de luxo adotado, política de sustentabilidade, FAQs ou outras secções informativas. O layout segue um padrão de cartões de informação (`.info-card` em `App.css`) organizados em grelha responsiva.

Componentes reutilizáveis

Para além das páginas, o projeto recorre a vários micro-componentes que permitem uma forte reutilização de código e um design consistente.

ProductCard.jsx

- `ProductCard.jsx`: representa um cartão individual de produto. Exibe:
 - Imagem do produto com overlay e pequenos efeitos ao hover.
 - Categoria, nome, cor e preço, incluindo possibilidade de mostrar preços anterior e atual (caso de promoções).
 - Botão ou ícone para marcar/desmarcar como favorito e link para a `ProductPage` correspondente.

Este componente é utilizado tanto em `ProductGrid` como em listas específicas (favoritos, resultados de pesquisa), contribuindo para a uniformização da experiência.

ProductGrid.jsx

- `ProductGrid.jsx`: componente que recebe um array de produtos e os apresenta numa grelha responsiva, delegando em `ProductCard` o rendering de cada item. Permite aplicar lógica de paginação, ordenação ou filtros a montante, mantendo a grelha em si focada apenas na parte visual.

Pagination.jsx

- `Pagination.jsx`: encapsula os controlos de paginação (botões anterior/seguinte, números de página atuais e totais) e emite callbacks quando o utilizador navega entre páginas de resultados. É reutilizado em contextos onde o número de produtos justifica a divisão em várias páginas, como nas categorias principais.

CategoryToolbar.jsx

- `CategoryToolbar.jsx`: barra no topo das páginas dos produtos, que agrupa:
 - Informações sobre a categoria atual.
 - Filtros de alto nível (por exemplo, ordenação por preço ou novidades).
 - Botão para abrir a `FilterSidebar` em dispositivos móveis.

Esta toolbar contribui para a usabilidade ao concentrar controlos relevantes no mesmo local.

categoryFilters.jsx

- `categoryFilters.jsx`: ficheiro auxiliar que define a configuração dos filtros disponíveis por categoria (intervalos de preço, cores pré-definidas). Em vez de codificar opções diretamente no JSX, esta configuração é externalizada, tornando o sistema mais fácil de manter e estender.

FilterSidebar.jsx

- `FilterSidebar.jsx`: componente responsável por criar o painel lateral de filtros, com categorias expansíveis (tamanho, cor, preço) e checkboxes para cada opção. Integra com o `StoreContext` para aplicar filtros e atualizar o conjunto de produtos visíveis em `CategoryPage`.

Newsletter.jsx, FooterColumns.jsx, FooterBottom.jsx

- `Newsletter.jsx`: pequeno componente que faz o formulário de subscrição de newsletter no rodapé, com campo de email e botão
- `FooterColumns.jsx`: organiza o conteúdo em várias colunas temáticas, cada uma com os seus links de redirecionamento ou páginas.
- `FooterBottom.jsx`: componente que exibe a zona inferior com direitos de autor, o ano corrente e os termos e cache.

Todos estes componentes são combinados dentro de `Footer.jsx` para construir um rodapé rico e coeso.

Funcionamento global e fluxo de dados

O fluxo de dados na aplicação segue um padrão previsível:

1. O `StoreProvider` (em `StoreContext.jsx`) é inicializado no `main.jsx`, carregando os dados de `storeData.js` para o estado base.
2. O `App.jsx` define o layout global e as rotas, garantindo que `Header` e `Footer` estão sempre presentes.
3. As páginas (`HomePage`, `CategoryPage`, `ProductPage`, `CartPage`, `FavoritesPage`, `CheckoutPage`, `AccountPage`, `SearchPage`, `InfoPage`) consomem os dados do contexto e invocam métodos de negócio para

atualizar o estado (por exemplo, adicionar ao carrinho, marcar favorito, aplicar filtros, registrar utilizador).

4. Os componentes reutilizáveis (ProductGrid, ProductCard, Pagination, FilterSidebar, CategoryToolbar, etc.) recebem dados e callbacks via props, focando-se na apresentação e delegando a lógica mais complexa para o contexto e para as páginas.

Esta separação permite cumprir o objetivo de alta performance, uma experiência sem recarregamentos de página e uma arquitetura escalável para evoluir o projeto.

5. Resultados e Discussão

A implementação realizada permite concluir que os objetivos centrais da primeira fase do projeto foram, em larga medida, atingidos. A aplicação apresenta uma estrutura sólida de SPA, navegação funcional entre múltiplas páginas, componentes reutilizáveis, catálogo navegável e um conjunto relevante de funcionalidades associadas a uma loja online moderna.

Do ponto de vista da experiência do utilizador, os resultados são positivos em vários aspetos. A homepage cumpre bem o papel de entrada visual e comercial para o catálogo, as páginas de categoria organizam a exploração dos produtos de forma clara, e a página de detalhe oferece ações diretas e coerentes com o fluxo esperado de compra. O uso de filtros, pesquisa, carrinho e favoritos reforça a sensação de completude funcional da interface, mesmo sem existir uma componente backend real nesta fase.

Em termos de organização interna, a separação entre páginas, componentes, dados e contexto acabou por parecer adequada. Esta estrutura torna o projeto mais legível, mais fácil de manter e mais preparado para uma possível evolução futura, nomeadamente integração com APIs ou quando se fizer o backend real.

Avaliação crítica

Apesar dos resultados globalmente positivos, uma avaliação crítica mostra que o projeto funciona sobretudo como um protótipo front-end avançado, e não ainda como uma solução completa de comércio eletrónico em produção. A aplicação simula várias operações importantes — autenticação, favoritos, checkout,

persistência de sessão — mas fá-lo com base em armazenamento local e dados estáticos, o que é adequado ao contexto académico, mas limitado para cenários reais.

Do ponto de vista arquitetural, a opção por Context API é adequada à escala atual do projeto. No entanto, à medida que o número de funcionalidades crescer, poderá tornar-se vantajoso adotar uma solução com maior segmentação de estado ou uma arquitetura de dados baseada em chamadas assíncronas e cache, como React Query, Zustand ou Redux Toolkit.

Também é importante notar que alguns componentes importam diretamente Header e Footer ao nível das páginas, enquanto o `App.jsx` desempenha simultaneamente um papel central de composição e routing. Isto funciona, mas também podia ser benéfico uniformizar Header e Footer layout partilhado único para evitar repetição e facilitar manutenção.

Limitações

Entre as principais limitações observadas destacam-se:

- Ausência para já do backend real, o que impede operações persistentes em servidor, autenticação segura e encomendas reais.
- Utilização de dados estáticos em `storeData.js`, limitando escalabilidade e atualização dinâmica do catálogo.
- Persistência local de utilizadores e sessões no navegador, suficiente para demonstração, mas inadequada para segurança real.
- Inexistência de integração com API de pagamentos, cálculo logístico real de portes ou validação externa de moradas e dados fiscais.
- Dependência de imagens e conteúdos locais/estáticos, sem pipeline de gestão remota de media.
- Falta de testes automatizados e ausência de uma camada explícita de validação avançada, tratamento de erros ou monitorização.

Estas limitações não invalidam o trabalho realizado; antes mostram que a solução foi desenhada com o foco correto para uma fase inicial de front-end, priorizando arquitetura, usabilidade e coerência visual.

Comparação com objetivos

Comparando a implementação com os objetivos inicialmente definidos, verifica-se uma correspondência clara entre o planeado e o concretizado. O objetivo de

construir uma SPA modular foi cumprido através do uso de React, React Router e Context API. O objetivo de oferecer uma experiência de navegação por catálogo foi atingido com a homepage, categorias, subcategorias, filtros e detalhe de produto.

Também foram concretizados alguns objetivos associados à interação do utilizador, como carrinho, favoritos, pesquisa e área de conta. Já os objetivos mais avançados de uma plataforma completa, autenticação robusta, base de dados real, pagamentos, administração de catálogo e integração cloud, ficaram para quando se fizer o back-end já completo.

5. Conclusões e Trabalho Futuro

A primeira fase do projeto permitiu construir uma base funcional, organizada e visualmente consistente para uma aplicação de e-commerce desenvolvida em React. A solução demonstra domínio de conceitos fundamentais de front-end moderno, como componentização, gestão de estado global, persistência local e responsive design.

Do ponto de vista académico, o projeto cumpre bem o propósito de mostrar capacidade de análise, planeamento e implementação de uma interface relativamente complexa, com múltiplos fluxos e componentes interdependentes. A organização interna do código também revela preocupação com escalabilidade e legibilidade, o que é importante numa aplicação com várias páginas e comportamentos partilhados.

Conclusões

Conclusões gerais

Em termos gerais, conclui-se que a aplicação já oferece uma experiência bastante próxima da de uma loja online real no plano visual e interativo. O utilizador pode navegar pelo catálogo, consultar produtos, pesquisar, aplicar filtros, gerir favoritos, construir um carrinho e simular uma finalização de compra, tudo isto num ambiente consistente e sem transições abruptas entre páginas.

A escolha das tecnologias mostrou-se apropriada para os objetivos da entrega. React e React Router garantiram uma navegação flexível e modular, enquanto a Context API e o armazenamento local permitiram simular estado persistente sem dependência imediata de backend.

Melhorias possíveis

Como evolução natural do projeto, destacam-se as seguintes melhorias possíveis:

- Integração com uma API ou backend real para produtos, utilizadores e encomendas, substituindo a dependência de `storeData.js` e `localStorage`.
- Implementação de autenticação segura com hashing de palavras-passe e sessões controladas no servidor, em vez de persistência local simples.
- Ligação a um serviço cloud de base de dados e autenticação, como Firebase ou Supabase, para suportar persistência multiutilizador e sincronização entre dispositivos.
- Adição de testes unitários e de integração para componentes críticos, como carrinho, filtros, autenticação e checkout.
- Melhoria da acessibilidade, incluindo navegação por teclado mais extensa, atributos ARIA adicionais e validação de contraste e foco em todos os elementos interativos.
- Introdução de uma área administrativa para gestão de produtos, categorias, stock e campanhas promocionais.
- Otimização adicional de performance com lazy loading de páginas, code splitting e eventual separação mais fina do estado global.
- Integração com métodos de pagamento simulados ou reais, bem como geração de encomendas e histórico de compras na conta do utilizador.

Referências

Meta Platforms, Inc. (n.d.). *React documentation*. React. <https://react.dev>

Meta Platforms, Inc. (n.d.). *React reference overview*. React. <https://react.dev/reference/react>

Vite Team. (n.d.). *Vite: Getting started*. Vite Documentation. <https://v3.vitejs.dev/guide>